

Uy Nguyen

Explanation: My program firstly creates a thread pool(consumers) based on user's demand that awaits work, and a listener thread(producer) that keeps inserting new requests into the buffer and listens indefinitely. Each worker thread will be put in a loop to work indefinitely. The main thread will monitor the number of worker threads in the pool. The main thread will recreate new thread if any thread is terminated prematurely. Two semaphores(sem_empty = 100 and sem_full = 0) and one mutex(lock) will be used to prevent race conditions. The producer or the listener thread, when wanting to enter critical section, has to check for any empty slots or wait, by calling sem_wait(&sem_empty). If there is some slots, the producer will call pthread_mutex_lock(&lock) to enter the critical section and insert request to the buffer. Then, it unlocks mutex and signals consumers the new availability. On the other hand, the consumers or the worker threads, when entering the critical section, have to check for any available requests by calling sem_wait(&sem_full). If there is requests, the consumers will call pthread_mutex_lock(&lock) to check if they can enter the critical section to get requests or wait. After successfully getting requests, worker threads unlock mutex, signal producer to proceed, and process requests. The main thread will identify prematurely terminated thread and respawn new thread accordingly. Each worker thread is assigned to a number to be tracked and monitored. The program implements the producer-consumer model to prevent race conditions and deadlocks. I've run the program multiple times, and there is no race conditions and deadlocks. Moreover, I modified client.c and net.c file to make them compatible to MacOS, and I included them in the submitted file.